

Pipeline Parallelism, from scratch

How `picotron`'s `pipeline_parallel` works — then the last two years of PP research

Why PP → naive pipelining → the bubble → 1F1B → **Zero-Bubble, Interleaved 1F1B**

build up from "split the model across devices" to two modern schedules that each attack the `(p-1)/m` bubble

deep dive: micro-batching · autograd B/W split · virtual stages · deadlock-free P2P · bit-exact tests

Why pipeline parallelism?

A large model's parameters + activations + optimizer state don't fit on one device.
picotron already

shards along **data (DP)**, **tensor (TP)**, **context (CP)** and **expert (EP)** axes.

Pipeline

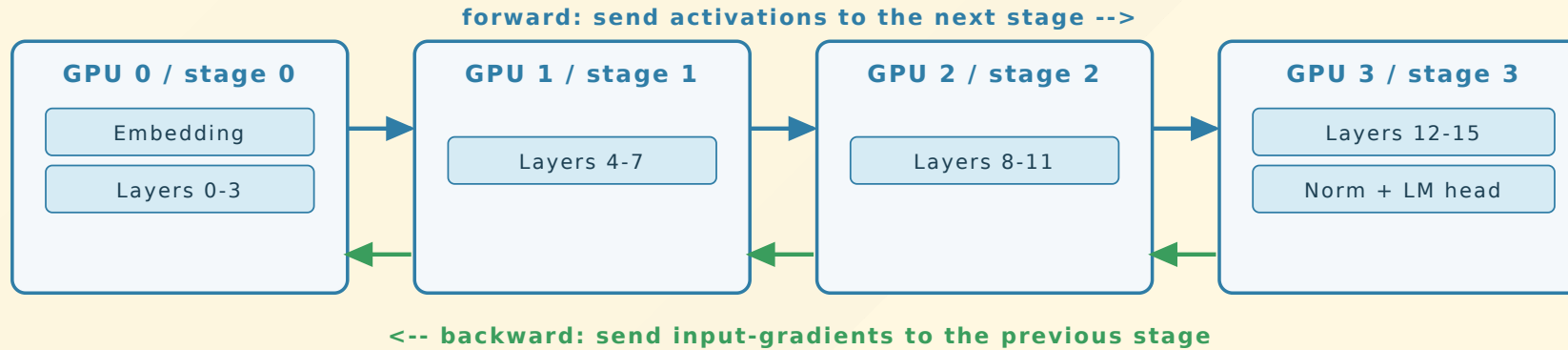
parallelism (PP) adds another: split the model **by depth** — give each device a contiguous block of layers (a *stage*).

- **TP** splits *within* a layer → needs a fast all-reduce every layer (intra-node, NVLink).
- **PP** splits *across* layers → a device only sends its output activations to the **next** stage.
- That is a single point-to-point hop per stage boundary → cheap, and scales across nodes.

PP is how you train a model deeper than one node's memory with only neighbor-to-neighbor traffic.

Split the model into stages

Pipeline parallelism: split the model by depth, one stage per device



Each GPU talks only to its two neighbors (one point-to-point hop per boundary) -- cheap, scales across nodes.

Forward: stage s computes, sends activations to $s+1$. Backward: stage $s+1$ sends input-gradients

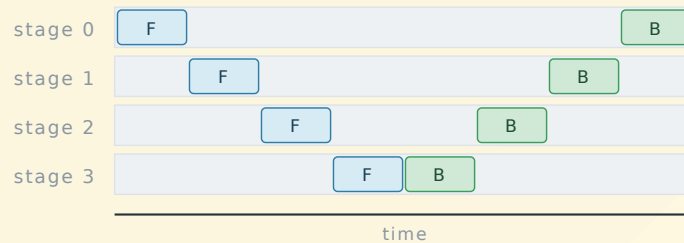
back to s . Only neighbors talk — that is

```
pipeline_communicate(send_forward / recv_forward / ...).
```

Naive execution wastes $p-1$ of every p devices

Run one batch straight through the stages: stage 0 works while 1, 2, 3 wait; then stage 1 while the rest wait; and so on — forward down the stages, backward back up.

Naive: one batch straight through -- only one stage ever busy



At any instant exactly **one** of p stages is doing work; the other $p-1$ wait.

utilization $\sim 1 / p$

$p = 4 \rightarrow 75\%$ of the cluster idle.

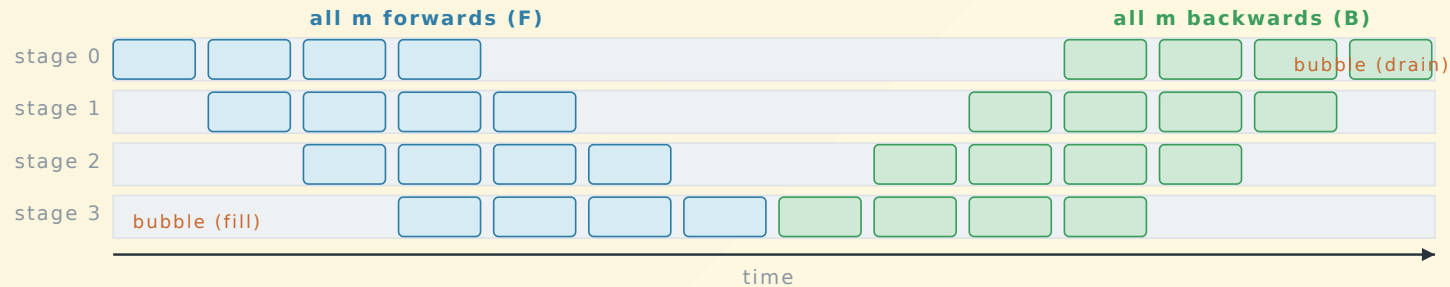
We need to keep all stages busy at once.

At any instant **one** of p stages is busy $\rightarrow$ utilization $\sim 1/p$. With $p = 4$ we waste 75% of the cluster. We need all stages busy at once.

Micro-batching = pipelining (GPipe / AFAB)

Split the global batch into m **micro-batches** and feed them back-to-back. Now while stage 0 starts micro-batch 1, stage 1 works on micro-batch 0 — the stages **overlap**.

Micro-batching (GPipe / AFAB): m micro-batches pipelined, $p=4$ $m=4$



Stages now overlap (good!), but AFAB must **keep every micro-batch's activations alive until backward** -> memory
The grey triangles at the start (fill) and end (drain) are the pipeline bubble.

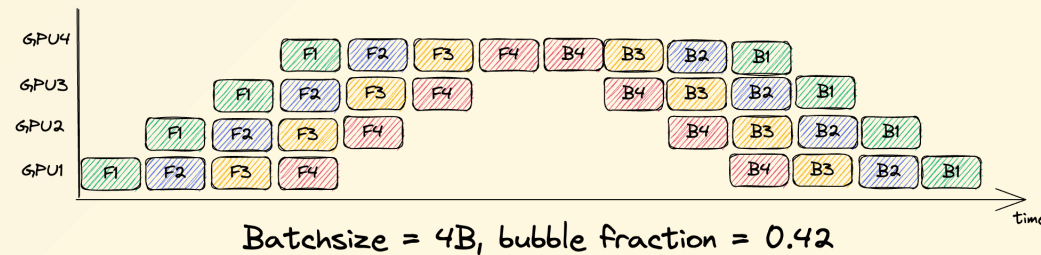
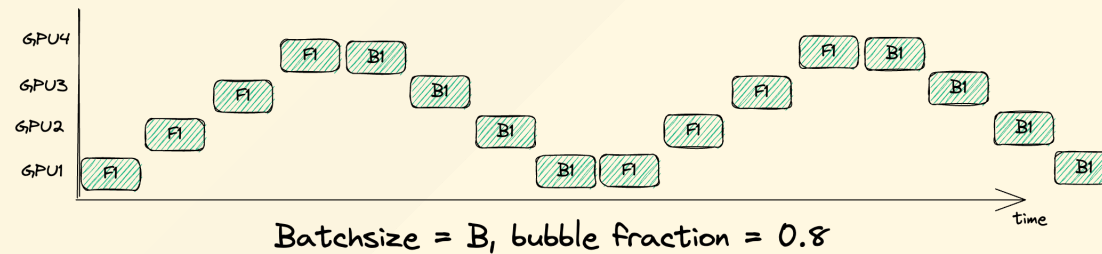
AFAB (all-forward-all-backward) = picotron's `train_step_pipeline_afab`: push all m forwards,
then all m backwards. Simple — but it keeps **every** micro-batch's activations alive until backward,
so activation memory grows with m .

The bubble, quantified

The pipeline still needs $p-1$ steps to **fill** and $p-1$ to **drain** — the shaded triangles where

some devices idle. Relative to the m useful steps:

$$\text{bubble fraction} = \frac{\text{idle}}{\text{busy}} = \frac{p-1}{m}$$

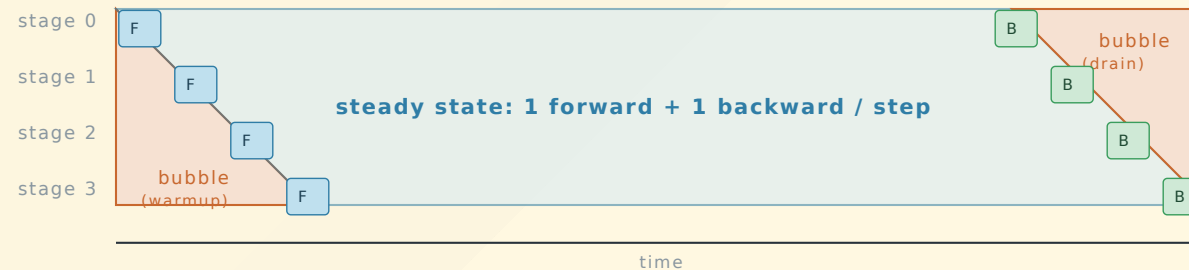


- More micro-batches ($m \uparrow$) $\rightarrow$ thinner triangles $\rightarrow$ smaller bubble (above: $m=1 \rightarrow 0.8$, $m=4 \rightarrow 0.42$)... but AFAB's activation memory $\uparrow$.

1F1B: same bubble, bounded memory

Reorder so each stage, after a short warmup, does **one forward then one backward** in lockstep. Each backward frees the activations of the oldest in-flight micro-batch, so a stage holds only $\sim p$ micro-batches' activations (not m).

1F1B: the pipeline bubble is the staggered fill & drain



Bubble fraction (fraction of time a device is idle):

$$\text{bubble} = (p - 1) / m$$

p = #stages, m = #micro-batches. More micro-batches $\rightarrow$ thinner triangles.

Same $(p-1)/m$ bubble as AFAB, but activation memory is capped by pipeline **depth**, not m (e.g. $m=100$: AFAB holds $\sim 100\times$ activations, 1F1B only $\sim p$). This is picotron's default (`train_step_pipeline_1f1b`) and the foundation everything below builds on.

picotron's two built-in schedules — and our goal

```
# pipeline_parallel.py
train_step_pipeline_afab  # all forwards, then all backwards - biggest memory + bubble
train_step_pipeline_1f1b  # 1 fwd / 1 bwd steady state - bubble (p-1)/m, bounded memory
```

Each rank holds a `PipelineParallel` stage: `embedding` (first), some `decoder_layers`, `final_norm` + `final_proj` (last). Communication is point-to-point on a ring.

Goal: shrink the $(p-1)/m$ bubble. Two levers, two trade-offs:

schedule	idea	trades
Zero-Bubble	split backward into B + W	W-queue activation memory ($\sim 1 \times$ FLOPs)
Interleaved 1F1B	v virtual stages per rank	$v \times$ more comm

The levers in one picture

1F1B



Time →



Forward



Backward



Optimizer step

ZB1P



Time →



F



B

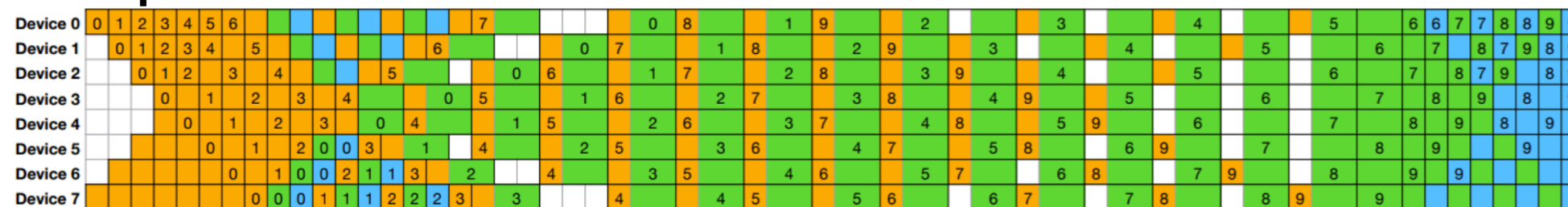


W



Optimizer step

DualPipe



Time →



Forward



Backward



Backward for input



Backward for weights



Overlapped forward & Backward

DeepSeek-V3 (arXiv:2412.19437). **Top — 1F1B**: the $\frac{(p-1)}{m}$ baseline. **Middle — ZB1P**: backward split into **B** (input-grad, teal) + **W** (weight-grad, green) → **Lever 1**. (Interleaved 1F1B / Lever 2 is the orthogonal " $\sqrt{v}$ chunks per rank" axis.) The **bottom**

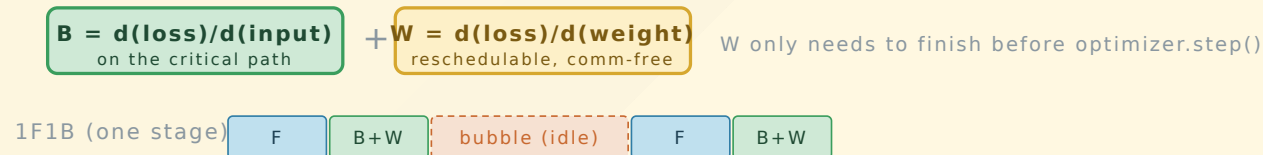
Lever 1 — Zero-Bubble (ZB-H1)

Qi et al., ICLR 2024 — arXiv:2401.10241

A backward fuses **B** (grad w.r.t. *input* — the previous stage needs it, so it's on the critical path) and **W** (grad w.r.t. *weights* — only needed before `optimizer.step()`, so reschedulable).

Zero-Bubble (ZB-H1): split backward into B and W, fill the bubble with W

A single backward fuses two computations:



The comm pattern is identical to 1F1B (B keeps the same send/recv), so it stays deadlock-free.
We split at the framework level with two autograd traversals □ bit-exact grads, ~2x backward FLOPs.
The win is bubble reduction on real clusters, not fewer FLOPs.

ZB: how we split the backward (true ~1x FLOPs)

We split at the **Linear** level, so each matmul runs **once** — not a 2x double-traversal. A custom autograd function (`_DeferredLinear`) returns the input-grad now and **defers** the weight-grad:

```
class _DeferredLinear(torch.autograd.Function):
    @staticmethod
    def backward(ctx, grad_y):
        x, weight = ctx.saved_tensors
        grad_x = grad_y.matmul(weight)           # B: needed upstream now (critical path)
        def _accumulate_weight_grad():           # W: queued, drained into the bubble later
            gw = grad_y.reshape(-1, D).t().matmul(x.reshape(-1, D))
            weight.grad = gw if weight.grad is None else weight.grad + gw
        _W_TASKS.append(_accumulate_weight_grad)
        return grad_x, None                      # weight-grad = None for now
```

`backward_input` runs autograd → collects only `grad_x` (+ a FIFO of deferred W-tasks per micro-batch);

`backward_weight` drains that FIFO. The **B** comm pattern is identical to 1F1B → deadlock-free, and

gradients are **bit-exact** (`grad_diff = 0`). Cost: the W queue holds `(x, grad_y)` per in-flight

micro-batch → extra activation memory (peak grows with `num warmup`, like ZB-H1 in the

Lever 2 — Interleaved 1F1B (virtual pipeline)

Narayanan et al., Megatron-LM — arXiv:2104.04473

Give each rank v **non-contiguous** chunks. With $G = p \cdot v$ virtual stages laid out round-robin,
each stage is $v \times$ smaller, so the fill/drain triangles are $v \times$ thinner.

Interleaved 1F1B (virtual pipeline): v chunks per rank, round-robin

8 layers, $p = 4$ ranks, $v = 2$ virtual stages $\rightarrow G = p \cdot v = 8$ virtual stages laid out round-robin:



activations flow vstage 0 $\rightarrow$ 1 $\rightarrow$ & $\rightarrow$ 7 (ranks visited 0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3)

Each rank owns 2 non-contiguous chunks:

rank 0 $\rightarrow$ layers {0, 4} rank 1 $\rightarrow$ layers {1, 5} rank 2 $\rightarrow$ layers {2, 6} rank 3 $\rightarrow$ layers {3, 7}

Each virtual stage is $v \times$ smaller, so the bubble shrinks by v :

bubble = $(p - 1) / (m \mid v)$ cost: $v \times$ more cross-rank activation hops (more comm). Needs $m \% p == 0$.

Interleaved: the wrapper + the schedule

```
class InterleavedPipelineParallel(nn.Module):  
    # virtual stage g lives on rank g % p, chunk g // p  
    self.chunk_global_id = [c * p + rank for c in range(num_virtual_stages)]  
    # embeddings / decoder_chunks / final_norms / final_projs are ModuleLists, one per chunk
```

- Per-chunk FIFOs of inputs/outputs/grads; `fwd_chunk(step)` / `bwd_chunk(step)` pick the active chunk.
- One **combined** ring send/recv per step (`interleaved_pipeline_communicate`): post all sends, drain receives first, then sends — the deadlock-free pattern for synchronous ring P2P.
- Requires `m % p == 0`.

Bubble: $(p-1)/(m \cdot v)$ · **cost:** v × more cross-rank activation hops.

Why batched P2P is mandatory

Synchronous ring P2P deadlocks if every rank blocks on a send whose matching receive sits behind the partner's own send. The fix used by every schedule here: fuse all of a step's sends+recvs into **one**

`batch_isend_irecv` group, which NCCL schedules together (and gloo posts non-blocking):

```
ops = []
if send_t is not None: ops.append(dist.P2POp(dist.isend, send_t, next_rank))
if recv_buf is not None: ops.append(dist.P2POp(dist.irecv, recv_buf, prev_rank))
for req in dist.batch_isend_irecv(ops): # one matched group → deadlock-free
    req.wait()
```

Every `send` is count-matched by exactly one `recv` in the schedule → no phantom op, no hang. (Posting unbatched `isend`/`irecv` made NCCL serialize them as collectives and hang at $p \geq 8$ — hence the batch.)

Correctness: bit-exact gradients vs a full-model reference

`tests/test_pipeline_parallel.py` builds a tiny Llama, takes a **non-pipelined** gradient reference on the same weights + micro-batches, shards the *same* modules across ranks, runs each schedule, and compares the gradients this rank owns.

```
torchrun --nproc_per_node 2 tests/test_pipeline_parallel.py # p=2
torchrun --nproc_per_node 4 tests/test_pipeline_parallel.py # p=4
```

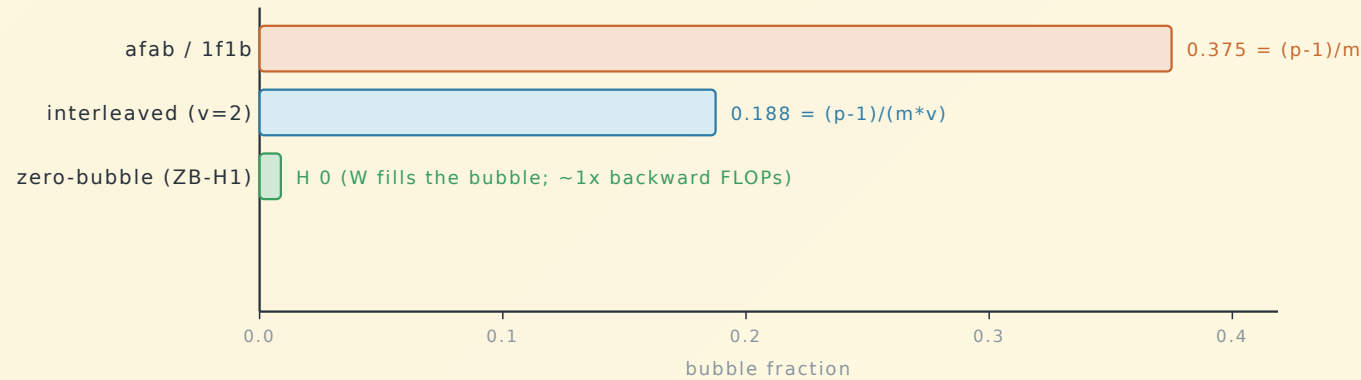
```
[rank 0] zero_bubble PASSED grad_diff = 0.00e+00
[rank 0] interleaved(v=2) PASSED grad_diff = 0.00e+00
```

ZB and interleaved match the reference **exactly** (`grad_diff = 0`) at `p = 2/4/8`.

The bubble, measured

Analytical bubble fraction ($p = 4, m = 8, v = 2$)

lower is better □ fraction of time a device sits idle in the steady state



`tests/bench_pp_schedules.py` reports measured step time next to these analytical fractions. On $8 \times L4$ ($p=8, bf16, m=16$): interleaved wins ($1.07\times$); ZB-H1's true $\sim 1\times$ split ties 1F1B (the per-Linear W-deferral's Python overhead roughly cancels the bubble-fill on commodity GPUs) while costing extra activation memory. An honest result. On a tiny CPU model the bubble is invisible (compute dominates).

Recap — two trade-offs against $(p-1)/m$

Zero-Bubble (ZB-H1)

- split backward $\rightarrow$ B (critical) + W (fill)
- bubble ≈ 0 , comm unchanged, $\sim 1\times$ FLOPs
- cost: W-queue activation memory

Interleaved 1F1B

- v virtual stages / rank
- bubble $(p-1)/(m \cdot v)$
- cost: $v \times$ comm, needs $m \% p == 0$

Both

- built on picotron's 1F1B engine
- deadlock-free ring P2P
- bit-exact vs full-model reference

Further reading: DualPipe (DeepSeek-V3)

- two opposing streams, $(p-1)/(2m)$
- $\sim 2\times$ param memory + custom CUDA overlap
- powerful but too complex for this module

Wired into `create_config.py --pp_engine` / `train.py`: ZB is a drop-in; interleaved runs on the test+bench harness (gloo/CPU + NCCL/GPU).

Thanks

`picotron/pipeline_parallel/pp_schedules.py` · `tests/test_pipeline_parallel.py` ·
`tests/bench_pp_schedules.py`

Zero-Bubble (Qi 2024) · Interleaved 1F1B (Narayanan 2021)